Pourquoi 0,1 + 0,2 n’est-il pas exactement égal à 0,3 ?

1. Une écriture binaire infinie

On part de $0,1 = 1,6 \times 0,0625$, donc $0,1 = 1,6 \times 2^{-4}$.

$$
\begin{aligned}
0,1 &= 1 \times 2^{-4} + (0,5 + 0,1) \times 2^{-4} \\
&= 1 \times 2^{-4} + 2^{-5} + 0,1 \times 2^{-4}
\end{aligned}
$$

En remplaçant à nouveau 0,1 dans le membre de droite :

$$
\begin{aligned}
0,1 &= 1 \times 2^{-4} + 2^{-5} \\
&\quad + (1 \times 2^{-4} + 2^{-5} + 0,1 \times 2^{-4}) \times 2^{-4} \\
&= 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 0,1 \times 2^{-8}
\end{aligned}
$$

On peut recommencer indéfiniment. Les chiffres binaires se répètent :

$$0,1_{10} = 0,0001100110011001100110011001100\ldots_2$$
$$0,2_{10} = 0,0011001100110011001100110011001\ldots_2$$

Ces écritures ne s’arrêtent jamais : il faut donc les arrondir pour les stocker en mémoire.

2. La précision utilisée par Python

Les nombres de type float sont généralement stockés selon IEEE 754 en double précision, avec 53 chiffres binaires significatifs (en comptant le premier 1).

Pour poser une addition exacte des valeurs effectivement stockées, on peut les écrire comme des entiers multipliés par une même puissance de 2 :

$$\texttt{float(0,1)} = 3602879701896397 \times 2^{-55}$$
$$\texttt{float(0,2)} = 7205759403792794 \times 2^{-55}$$

La seconde valeur est exactement le double de la première. Les deux valeurs stockées sont légèrement supérieures aux décimaux mathématiques 0,1 et 0,2.

L’addition binaire et son arrondi

3. Additionner les bits des deux nombres stockés

Les trois lignes ci-dessous sont des entiers binaires de 54 bits. Chaque ligne doit être multipliée par 2^{-55} pour retrouver sa valeur réelle.

```
  0011001100110011001100110011001100110011001100110011001101
+ 0110011001100110011001100110011001100110011001100110011010
  ----------------------------------------------------------
  1001100110011001100110011001100110011001100110011001100111
```

L’addition entière est exacte :

```
3602879701896397 + 7205759403792794
= 10808639105689191
```

4. Arrondir le résultat à 53 bits significatifs

Le résultat binaire possède 54 bits significatifs, alors que le format ne peut en conserver que 53. Il faut supprimer le dernier bit, qui vaut 1. On est exactement à mi-chemin entre deux valeurs représentables : la règle IEEE 754 « au plus proche, pair en cas d’égalité » conduit à augmenter le dernier bit conservé.

```
Somme exacte : 1001100110011001100110011001100110011001100110011001100111
Après arrondi: 1001100110011001100110011001100110011001100110011001101000
```

Ainsi, la somme stockée vaut :

$$10808639105689192 \times 2^{-55}$$
$$= 0{,}3000000000000000044408920985\ldots$$

5. Ce que Python affiche

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Python affiche une écriture décimale courte qui permet de retrouver exactement le flottant stocké. Le flottant correspondant à 0,3 écrit directement n’est pas le même que celui obtenu après l’addition.

À retenir : le calcul binaire est cohérent ; c’est la représentation finie des nombres et l’arrondi qui expliquent l’écart observé.